ElastOS Capsule System: Master Product Definition and PRD

For: Sash (ELA CITY), and through Bella for the team. **Prepared by:** Flint (Grok Bot), 20 Oct 2026, about 01:30 China time (UTC+8). **Status:** draft for Sash's review. Research and writing only. No repo, PR or issue was touched. No Cursor cloud agent or Claude API credit was used.

Sources. The runtime repo is `Elacity/elastos-runtime`, branch `develop`, commit `3beb8164` (committed 19 Oct 2026, 19:30 China time). I re-checked GitHub at 01:20 China time on 20 Oct and `develop` was still at `3beb8164`. Other sources: the 19 principles (`/workspace/ponytail-review/elastos_PRINCIPLES.md`, the same text as `PRINCIPLES.md` in the repo), Bella's meeting notes (`/workspace/meeting-2026-10-18/summary.md`, `/workspace/meeting-2026-10-19/summary.md`), my earlier research notes in `/workspace/research/`, the Elacity GitHub org (read only), and the public web. Every claim has a label:

Label	Meaning
[Code]	In the source tree. File path given.
[Doc]	Written in a repo document as a contract or plan. It may not be built yet.
[Org]	Read in another Elacity GitHub repo (most are private). Repo and path given.
[Web]	A public web page. URL given.
[Meeting]	Said in the 18 or 19 Oct Shanghai meetings (from transcripts, so names and numbers may be misheard).
[Proposal]	My recommendation. Not decided, not built.
[Estimate]	My rough judgement of effort or time.
[Unverified]	I could not confirm it.

Effort sizes are for a small team using AI agents: **S** = days, **M** = 2 to 6 weeks, **L** = 1 to 3 months, **XL** = a quarter or more. Owners are left as **TBD** on purpose. Durations are given relative to the end of Phase 1, not as calendar dates, unless a date comes from a source.

Executive summary (one page)

What we are building. ElastOS is a small trusted program, the **Runtime**, that runs signed, sandboxed packages called **capsules** (users see them as "Apps"). The Runtime holds all authority. A capsule can only do what the Runtime grants it, through narrow, signed, revocable permission tokens. Capsules are named by the hash of their contents, signed by their publisher, and carry their own small toolkit, so they don't depend on the host operating system. This matches Rong Chen's vision from the Shanghai meetings and our own Principle 18. The north star is: **one signed capsule, named by its hash, runs on any ElastOS device, with no ambient authority. Paid content is unlocked only by a key that is released to a Runtime that can prove what it is.**

Where we are (Phase 0). The pieces exist but are not joined up yet. The capsule manifest, roles, permission tokens, providers, Carrier networking and signed releases all work [Code]. But all 20 first-party apps are still web pages in sandboxed browser frames, providers run as normal programs with the user's rights, signing is per release rather than per capsule, and Elacity's dDRM code sits inside the MIT repo [Code, Doc].

The plan in four phases.

Phase	Goal in one line	Rough size [Estimate]
0 Today	Honest baseline: see section 4	-
1 Finish 0.8.0	alpha.25, fix #451, ship 0.8.0 (team target: end of October), move to the `elastos` org, update the launcher pin	weeks
2 WASM capsules on today's Runtime	Per-capsule signing by bundle hash and publisher ID; async Bus v2; Wasmtime 48; a capsule kit; move apps one at a time (first: a signage or photo viewer); move dDRM into an Elacity package; CDN on Carrier; metering receipts	about 6 to 9 months
3 Trusted hardware	A fast-booting, verified Linux image on one partner board; the Runtime measured into a TEE or secure element; keys sealed to the device; attested play counts	about 6 to 12 months, overlapping Phase 2

Sash's three questions, answered.

1. **Is it platform independent?** Yes, with caveats. A WASM capsule compiled once runs wherever our Rust Runtime with Wasmtime runs: Linux and macOS today, Windows through WSL2 later, and 64-bit ARM devices. The caveats: 32-bit ARM only gets a slower interpreter; ahead-of-time compiled code is tied to one CPU type (so we compile on the device at install); a device needs enough memory for the Runtime (cameras don't have it); providers stay native code per platform; and today's apps are web pages, not WASM yet (section 3).
2. **Does the player ship inside the content?** By default, **no**. A content package holds the encrypted assets, a manifest and a policy, and names its player by the player's hash (CID). The player is a separate, shared, signed capsule. It is **bundled** only when the content needs it to work offline or is tied to one engine, as most games are (section 8).
3. **Is dKMS still relevant? Yes. It becomes more important, not less.** The quorum is the layer that decides who gets a content key. In the new architecture it should release keys only to a Runtime that can prove it is genuine, and in Phase 3 only to a device whose key lives in a TEE or secure element. What changes is the recipient (an attested device key instead of a session key) and the evidence (an on-chain licence plus a device measurement) (section 9).

What I found about dKMS (short). Elacity has two generations. The **old path** uses Lit Protocol, now its "Chipotle" v3 service, with Lit Actions that call `AuthorityGateway.hasAccessByContentId` on Base [Org: `Elacity/lit-keystore-moleculer`]. The **new path** is Elacity's own 2-of-3 custody quorum, written in Rust inside the runtime repo: a Shamir split of the content key, post-quantum hybrid wrapping, three nodes that each check the licence on chain, and a decrypt provider that rebuilds the key [Code, Doc]. It is implemented in source but **not active** in the installed product [Doc: `docs/PROTECTED_CONTENT.md` L21–24]. Elacity's post-quantum roadmap plans to retire Lit and close the dKMS launch gates by roadmap week 38, which works out to about 12 Jul 2027 [Org: `Elacity/elacity-post-quantum-roadmap`].

Decisions only Sash can make (section 11.2): approve Phase 2 starting after 0.8.0; choose the first partner board and whether frames must be 64-bit; approve moving dDRM out of the MIT repo; pick "auditable" or "hardware-attested" as the play-count promise; decide whether Boson, NAS and expo items join this plan (they are still on hold).

---

1. Words used in this document

Term	Plain meaning
Runtime	The trusted ElastOS program (the `elastos` binary). It checks permissions, starts capsules, routes requests and keeps the audit log.
Capsule	A signed package the Runtime can run or open. Users call them **Apps**. Roles: `app`, `viewer`, `shell`, `provider`, `content` (P13, P14).
Provider	A capsule that does system work the Runtime trusts it with, such as networking, storage, keys or chain calls. Apps reach it only through the Runtime.
Capability token	A signed, short-lived, revocable permission slip from the Runtime: "this capsule may do this one thing to this one resource".
WASM (WebAssembly)	A portable, sandboxed form of compiled code. The same file runs on any CPU that has a WASM engine.
Wasmtime	The WASM engine we embed. Version 36 today. Version 48 is the newer long-term-support release.
Component Model, WIT	The WASM standard for typed interfaces. WIT is the small language that says exactly what a component may import (use) and export (offer).
WASI 0.3	The standard system interfaces for WASM. Version 0.3 (June 2026) adds built-in async, streams and futures.
Bus	Our own WIT interface: the **only** door from a WASM capsule to the Runtime. Bus v1 exists. Bus v2 is planned.
JIT / AOT	Just-in-time compiling (when the app starts) versus ahead-of-time compiling (once, at install).
Pulley	Wasmtime's interpreter. It runs where Wasmtime has no native compiler, such as 32-bit ARM. Slower.
CID	Content identifier: a name made from the hash of the bytes. Same bytes, same name. Different bytes, different name.
SHA-256	A standard hash function. A 256-bit fingerprint of data.
DID	Decentralised identifier. Ours are `did:key` values: an Ed25519 public key written as text.
Bundle CID	The CID of a whole capsule: its manifest plus every file it contains. Its one true name.
Carrier	ElastOS's peer-to-peer transport (Rust, built on iroh/QUIC). It moves bytes between devices. It grants no authority.
dDRM	Elacity's decentralised digital rights management: encryption, licences on chain, key release and royalties.
CEK	Content encryption key: the AES key that encrypts one asset.
dKMS	Distributed key management system: independent nodes that each hold one piece of a CEK and release it only when the rules are met.
Quorum / threshold, t-of-n	Any t of n nodes are enough to rebuild a key, and fewer than t learn nothing. Ours is 2-of-3.
TEE	Trusted execution environment: a hardware-isolated area of the chip (e.g. ARM TrustZone running OP-TEE) where code and keys are protected from the normal OS.
Secure element (SE)	A small separate security chip (e.g. Microchip ATECC608) that makes and keeps keys it never lets out.
Attestation	A signed statement from the hardware: "this device, running this exact measured software".

Term	Plain meaning
Measured / verified boot	Verified boot refuses to start unsigned software. Measured boot records a hash of each stage so it can be attested later.
dm-verity, A/B	dm-verity makes the root filesystem read-only and checks every block. A/B means two system copies, so an update can fall back to the old one.
BUSL-1.1	Business Source License 1.1: source is visible, commercial use is restricted until a change date, then it becomes open source.

---

2. First principles and the north star

2.1 The north star

> **One signed capsule, named by its hash, runs on any ElastOS device with no ambient authority. The Runtime that runs it is small enough to reason about and, on our own hardware, measured by the chip. Paid content is unlocked only by a key released by an independent quorum to a Runtime that can prove what it is. Creators are paid by contracts nobody can quietly edit.**

2.2 The fundamentals, from first principles

Strip the problem down and six facts remain. Everything else in this plan follows from them.

1. **A name must be the thing, not where it lives.** A URL says where bytes might be. A hash says what they are. So capsules, dependencies and content are named by CID and SHA-256 (P2, P15; Rong's ask).
2. **Authority must be handed out, never assumed.** If code can reach the network or disk just by running, nothing else matters. So capsules get only typed resources through capability tokens (P3, P18).
3. **A sandbox is only as good as what enforces it.** CAR components 20 years ago had the right ideas but shared one address space, so a bad component could break the rest. WASM gives a memory-safe sandbox that the Runtime enforces. So apps become WASM components (P18, section 2.5).
4. **Trust needs a root you can point at.** Today that root is a signed release. Next it is a publisher signature on each capsule. On our own hardware it is a key fused into the chip plus a measurement of the Runtime (P15, Phase 3).
5. **Whoever sees plaintext can copy it.** No DRM stops a determined owner of the hardware from copying what they can see. The honest goal is to keep keys out of apps, release them only to genuine Runtimes, and make copying expensive and detectable (P15, section 8).
6. **Money and counts must be checkable by people who don't trust each other.** Steve's factory partners don't need philosophy. They need a split no operator can edit and counts that are signed where they happen. So settlement lives on chain, and counts are signed in hardware where possible (P19, Phase 3).

2.3 How the 19 principles shape the capsule system

The principles are the law (`/workspace/ponytail-review/elastos_PRINCIPLES.md`). This table says what each one means for capsules and in which phase we satisfy it.

#	Principle	What it means for capsules	Main phase
1	Local first	Capsules start from the user's Home. Public copies derive from local state.	0 (holds)
2	Stable identity over transport	Capsules, dependencies and content are named by CID. Mutable heads point to immutable bundles. Availability needs signed receipts.	2
3	No ambient authority	A WASM capsule has no file, socket or clock unless the Runtime hands one in through the Bus.	2
4	Carrier is not the capsule contract	Capsules call `elastos://...` resources. Carrier, HTTP and loopback are hidden below.	0 (holds), 2 (CDN)
5	Small trusted core	The Runtime's authority code shrinks and is measured. Protocol work moves out to providers.	2, 3
6	Clear user, operator, developer boundaries	Users see Apps. Operators run custody nodes and boards. Developers use the capsule kit.	2
7	Humans and agents share one authority model	An agent capsule gets the same tokens and audit as a person.	0 (holds)
8	WebSpaces resolve names dynamically	A capsule's view of data comes from grants, not from paths.	2
9	HTTP is edge transport	Web gateways only proxy. Capsule identity is the bundle CID, not a URL.	2
10	One canonical path per operation	One admission path, one key-release path (the Lit path is retired; the provisional `key-provider` is removed).	1, 2
11	Fail closed, then explain	An unsigned or unknown capsule is refused with a clear reason. No silent downgrade.	2
12	Docs, code, tests and operations agree	Every phase exit needs tests and docs that match.	all
13	Objects, capsules and spaces stay distinct	Photos are objects. A game is a content capsule. A player is a viewer capsule. A store row is not identity.	2
14	Public names match human mental models	"Apps", not "capsules", in the product.	all
15	Trust and access travel with signed content	Publisher DID signatures, CIDs, explicit decrypt provider, dKMS over Carrier.	2, 3
16	UI surfaces are not authority	Opening a frame is not holding a permission. Home stops holding every app's token.	2
17	Design tokens are product contracts	A shared look and feel across capsules.	2
18	Executable capsules are isolated environments	Each capsule brings its app, language runtime and a minimal system surface. The host OS is not its ABI. This is Rong's vision in our own words.	2
19	Consequences govern effects	Payments, play counts and device control are classed by consequence, not by which wire carried them.	2, 3

2.4 The decision rule applied to the big choices

When two options both work, we pick the one that (1) strengthens rooted and content identity, (2) reduces ambient authority, (3) removes hidden alternate paths, (4) keeps the trusted core smaller and (5) makes the user model clearer.

Choice	Picked [Proposal]	Why, by the decision rule
What runs inside a capsule	WASM components on Bus v2. No classic RTOS inside.	(2) no ambient authority; (4) one scheduler, in the Runtime; (5) one model for every app
What runs under the Runtime on devices	A small verified Linux, not an MCU RTOS	(4) reuses our existing binary; (3) one boot path
How capsules are named	Bundle CID plus publisher DID signature	(1) content identity; (3) no URL fallback
How a player relates to content	Player referenced by CID; bundled only when needed	(1) one player identity shared by all content; (4) fewer copies of decrypt-adjacent code
Where keys are rebuilt	Only in the decrypt provider; in Phase 3 only inside the TEE	(2), (4) the smallest place that ever sees a key
Old Lit key path	Retire it	(3) removes a weaker alternate path (P10, P11)
dDRM code location	An Elacity-owned provider bundle	(4) a smaller open core; (5) a clear licence split
"Baked on the chip"	Hardware root of trust plus a measured Runtime. No custom silicon now.	(4) small trust base; cost and risk (section 7.1)

2.5 Rong Chen's vision, and the COMO/CAR lesson

What Rong asked for [Meeting] (`meeting-2026-10-18/summary.md` §1 Rong Chen and §3; `meeting-2026-10-19/summary.md` §2A, §2E, §4):

1. Every app is a **digital capsule** that carries its own small runtime pieces, so the host OS is not its interface.
2. A small user-mode **micro-kernel** (he said about 10k lines) that only multiplexes capabilities: "one web space can't write another without capability tokens".
3. A **signature and hash on every module dependency**, checked recursively, using **CIDs instead of URLs** and **SHA-256 instead of 128-bit UUIDs**, with DNS- or DID-style names for people to read.
4. Keep the interface language simple and strip 25 years of cruft.
5. Burn the base into firmware for **instant-on**, so only capsules run.
6. Content is fetched by address, so home boxes and cameras become a CDN, and play counts are self-validated ("no YouTube lie").

Where we already agree [Doc]. P18 says almost exactly item 1. P3 and P4 cover item 2. P2 and P15 cover items 3 and 6. Item 5 is not in the repo; it is Phase 3 here.

The COMO/CAR lesson [Web, Doc]. Rong's earlier Elastos (和欣, Koretide, 2000s) used **CAR** components: self-describing binary components, runtime-generated proxies that controlled access, and components named by a unique network name, downloaded on demand, cached and signed. **COMO** (`gitee.com/tjopenlab/como`, Apache-2.0, C++, DBus) is its current descendant. Sources and detail: `/workspace/research/rtos-recommendation.md` §5. Our team decided in June 2026 to "adopt the COMO lesson, not the COMO runtime" (`docs/RUNTIME_FRAMEWORK_RESEARCH.md`, on branches `flint-0.5` and `feat/shell-ui-v1`, not on develop). What CAR lacked was a memory-safe sandbox: native code in a shared address space can't be confined. The modern, enforceable version is a one-to-one map:

CAR idea (then)	ElastOS capsule (now)
Self-describing component metadata	WIT world plus `wit_world_sha256` in the manifest
Runtime-generated proxy that checks access	The Bus: every call is checked against a capability token
Unique network name (`uunm`)	Bundle CID (content-addressed, so it can't be spoofed)

CAR idea (then)	ElastOS capsule (now)
Digital signature on the component	Publisher DID signature over the bundle CID
Download on demand, cache locally	Fetch by CID over Carrier, AOT-compile and cache by CID
Native code, shared address space	WASM sandbox with fuel, memory and time limits

The one-line story for Rong: "ElastOS capsules are CAR, made safe."

---

3. Plain answer: does a WASM capsule run everywhere?

Yes, mostly. WASM is compiled once into a portable format. Any machine with a WASM engine can run it. Our Runtime embeds Wasmtime, so wherever the Runtime runs, the same capsule file runs, with no recompiling by the developer.

```
developer builds once  ->  capsule.wasm (+ manifest, signed, named by bundle CID)
                                  |
      +---------------------------+---------------------------+
      |                           |                           |
 Mac (arm64)              Linux PC / server            ARM NAS / 64-bit frame
 elastos + Wasmtime       elastos + Wasmtime           elastos + Wasmtime
 (compiles for Apple      (compiles for x86_64         (compiles for aarch64
  silicon at install)      at install)                  at install)
```

The real caveats:

Caveat	What it means	Source
Today's apps aren't WASM yet	All 20 first-party apps are web pages in sandboxed frames. The WASM path is built and tested but no shipped app uses it. Phase 2 fixes this.	[Code/Doc] `docs/CAPSULE_AUTHORING.md` L13–16
Where the Runtime itself runs	Release builds exist for x86_64 Linux, aarch64 Linux (musl) and aarch64 macOS. Windows is not a native target; the plan is WSL2 first. No 32-bit ARM or MIPS build.	[Code] `.github/workflows/release-package.yml` L376–380; [Doc] `docs/WINDOWS.md`
32-bit ARM gets only the Pulley interpreter	Wasmtime's Cranelift compiler has no 32-bit target. Many cheap frames are 32-bit ARMv7, so capsules would run interpreted, which is slower. Ask partners for 64-bit chips.	[Web] https://docs.wasmtime.dev/stability-platform-support.html
AOT-compiled code is tied to one CPU	We ship the portable `.wasm`. Each device compiles it once at install and caches the result by CID. A precompiled file can't be copied to a different CPU type.	[Proposal] `rtos-recommendation.md` §3.2
A device needs enough memory	The Linux arm64 release tarball was 55.8 MB compressed at v0.7.1. Cheap IP cameras have 64 to 128 MB of RAM, so cameras should be clients of a nearby NAS, not Runtime hosts.	[Web] GitHub release v0.7.1; `rtos-recommendation.md` §2
Providers stay native per platform	Drivers, GPU inference, codecs, Carrier and the browser engine are native code compiled per platform, behind typed provider interfaces. Apps don't see the difference.	[Code/Doc] `docs/CAPSULE_MODEL.md` L19–21
The sandbox doesn't make the host trustworthy	WASM protects the device from the capsule. It doesn't protect the capsule's secrets from the device owner. That needs a TEE (Phase 3).	first principles, §2.2 item 5

---

4. Phase 0: today, the honest baseline

All facts at `develop` `3beb8164` unless marked. Detail: `/workspace/research/capsule-architecture.md`.

Area	What exists	Gap	Where
Capsule manifest	`elastos.capsule/v1`, strict (unknown fields rejected). Fields for role, ABI, Bus contract, WIT hash, requested capabilities, interfaces, resources, microVM, viewer, model metadata, optional signature.	The signature field isn't checked at launch.	[Code] `elastos/crates/elastos-common/src/manifest.rs` L14–133, L407, L432+
Roles	`app`, `viewer`, `shell`, `provider`, `content`. Providers declare exactly one namespace plus full authority metadata.	-	[Code] `manifest.rs` L1128–1163; [Doc] `docs/CAPSULE_AUTHORING.md` L58–70
What actually runs	20 first-party apps are web projections in opaque sandboxed iframes. About two dozen providers are native processes. The browser engine runs in a microVM.	WASM path unused. "Home can currently obtain every app's capability." Providers run "with the Runtime user's rights".	[Doc] `docs/CAPSULE_MODEL.md` L12–21; `CAPSULE_AUTHORING.md` L13–16
Capability tokens	Ed25519-signed tokens bound to a capsule, one resource, one action, with expiry, epoch, max uses; grant, validate, delegate, revoke, key rotation; pending requests for user approval; pluggable policy.	Home still mints every app's token (issue #173, T2/T3).	[Code] `elastos-runtime/src/capability/token.rs` L185–230, `manager.rs` L309–605, `pending.rs`, `policy.rs`
Providers	Line-delimited JSON over stdin/stdout, 30 s timeout. Reserved Runtime-only targets: protect, media, custody, protected-content-decrypt.	Only the model provider is partly confined (seatbelt/seccomp).	[Code] `elastos-runtime/src/provider/bridge.rs` L1–5, L29, L634; `registry.rs` L653, L682
Bus v1 (WASM)	World `product-capsule-v1`: `runtime.info`, `identity.context`, `capabilities.request`, `providers.invoke`; export `lifecycle.run`. Wasmtime 36.0.18. Fuel 100,000,000, memory 128 MiB. No WASI.	Run-once, synchronous, no UI or streams. No wall-clock limit. No compile limit (#440). Bus v2 is #181.	[Code] `elastos/wit/elastos-bus-v1.wit` L90–96; `elastos-compute/src/providers/component.rs` L28–29, L83–91
Carrier	Rust module on iroh 1.0.2 (QUIC), iroh-gossip 0.101.0, mDNS. Node ID = Ed25519 `did:key`. Typed providers for peers and content.	One community seed (single point of failure).	[Code] `elastos-server/src/carrier.rs` L1–19; `elastos-server/Cargo.toml` L110–115; `collaboration-network-release-v1.json`
Signing	A signed release envelope checked against publisher DIDs, then every binary and component checked by SHA-256.	Per release, not per capsule. A capsule fetched by CID has "no fetched signature-based trust model". Publisher root key (#213) and third-party capsules over Carrier (#217) open.	[Code] `elastos-server/src/crypto.rs` L186–201; `update_controller.rs` L1559–1615; `capsule/fetched_launch.rs` L16–20

Area	What exists	Gap	Where
Trusted core size	`elastos-runtime` kernel crate about 9,900 lines of code.	It is linked into one binary with `elastos-server` (about 200,000 lines) that also holds authority.	[Measured] `capsule-architecture.md` §1.9
dDRM in the MIT repo	Protect, media, custody and decrypt providers; five `elastos-protected-content-*` crates (about 23k lines); the coordinator compiled into `elastos`; `chain-provider` with Elacity's Base 8453 calls. Workspace licence is MIT.	Elacity's protocol code sits in the open repo.	[Code] `LICENSE`; `capsules/chain-provider/src/main.rs` L40–56; `elastos-server/src/protected_content_runtime/`
Protected content status	The canonical 2-of-3 path is "implemented but inactive". The installed product still uses provisional `drm`, `rights`, `key`, `decrypt` surfaces, which fail closed.	Installed proof and an atomic cutover remain. External crypto review open (#48).	[Doc] `docs/PROTECTED_CONTENT.md` L21–24, L153–156, L298–330; [Code] `capsules/key-provider/src/main.rs` L130–141
Content capsules	Content role exists, with a viewer named in the manifest (e.g. `gba-ucity` opened by `gba-emulator`).	The viewer is named by name, not by CID.	[Code] `capsules/gba-ucity/capsule.json` L3–11

In one sentence: the authority model is real and good, the isolation and signing are half done, and the dDRM layering still has to be drawn.

---

5. Phase 1: finish the 0.8.0 line

Goal. Ship a stable 0.8.0 that people can install and that stays connected, before any Phase 2 work starts. Nothing in Phase 2 should delay this.

State now. The head commit says "CI update journey starts from 0.8.0-alpha.23, the release users run now" (`3beb8164`). `elastos/CHANGELOG.md` already has a `[0.8.0-alpha.24]` release-candidate section dated 2026-10-19 and a drafted `[0.8.0]` section. The newest version tag on GitHub is `v0.7.1` (checked with `git ls-remote` at 01:20 on 20 Oct). The plan below comes from the team's own plan as relayed in our conversation; the dates are team targets, not repo facts.

Step	What	Exit criteria	Effort	Owner	Depends on
1.1	**alpha.25** test build	Published on the test channel. The CI update journey passes from alpha.23.	S	TBD	-
1.2	**Fix #451**: Community stays unreachable after a network change until Home restarts (open, updated 19 Oct 2026, 23:15 China time). Cause seen in the logs: gossip rebuilds capped at 6 per process.	A Home that changes network (e.g. Wi-Fi to hotspot) shows Community reachable again without a restart. A test covers it.	S	TBD	-
1.3	**0.8.0 stable** (team target: end of October)	Signed release published. Release notes match behaviour (P12). Isolation status written honestly, as the changelog already does.	S	Anders merges; the founder decides the release	1.1, 1.2

Step	What	Exit criteria	Effort	Owner	Depends on
1.4	**Move the repo to the `elastos` org**	GitHub serves the repo under `elastos/` with redirects. The updater, launcher and docs point at the new home. Release signing keys unchanged.	S	TBD	1.3
1.5	**Launcher pin** (the launcher was on alpha.22 at our last check, per the conversation)	The launcher installs 0.8.0 from the new org, or reads the latest stable release, which is fine once 0.8.0 is published.	S	TBD	1.3, 1.4

Phase 1 exit: 0.8.0 stable is published from the `elastos` org, #451 is closed, and the launcher installs it.

---

6. Phase 2: WASM capsules on the current Runtime

Goal. Move what we've built into the capsule model Rong described, **on the hosts we already support** (Mac and Linux, plus aarch64 NAS boards). No new hardware is needed. At the end, at least one real product app runs as a signed WASM capsule, admitted by its bundle CID and publisher DID, talking to the Runtime only through Bus v2. Elacity's dDRM runs as a separately licensed provider bundle.

G-numbers (G1, G3, ...) refer to the gaps in `/workspace/research/capsule-architecture.md` §3; #-numbers are GitHub issues on the runtime repo.

Why this order. Identity first (you can't sign, price or cache what you can't name), then the engine and the Bus, then the kit, then real apps. Confinement and dDRM extraction run in parallel. Metering and the CDN come last because they need stable names.

6.1 The capsule package after Phase 2 [Proposal]

```
capsule bundle (a gzip tar today; the bundle CID is the hash of the whole closure)
├── capsule.json              manifest: elastos.capsule/v2 (proposed), role, runtime_abi,
│                             bus_contract "elastos:bus@2", wit_world_sha256, requested
capabilities,
│                             interfaces, resources, and:
│     "requires": [            every dependency named by CID + SHA-256, never a URL or UUID
│        {"name":"elastos-guest-kit", "cid":"bafy...", "sha256":"..."},
│        {"name":"signage-player",   "cid":"bafy...", "sha256":"...", "role":"viewer"} ]
├── app.wasm                  the component (portable; AOT-compiled on the device at install)
├── ui/                       render-only UI assets (no authority)
└── (content capsules only)   encrypted assets + media manifest + rights-policy identity

signature record (separate, so the bundle CID stays stable):
   { bundle_cid, publisher did:key, Ed25519 signature over the bundle CID, issued_at, key epoch }
```

Admission rule (every install and launch path): recompute the bundle CID, check the publisher signature, check the DID against the local trust policy for this role, check every `requires` entry by CID and SHA-256 recursively (Rong's ask), then write a signed install receipt. Anything missing fails closed with an explanation (P11).

6.2 Milestones

#	Milestone	What it delivers	Exit criteria	Effort [Estimate]	Owner	Depends on
2.1	**Engine upgrade**	Wasmtime 36 to 48 LTS with component-model async; epoch interruption (a wall-clock limit, because fuel is not time); compile limits (#440); one shared engine; **AOT compile at install, cached by CID**	All current Bus v1 conformance tests pass on 48. A runaway capsule is stopped by time and by compile limits. Second launch uses the cache.	S–M (1–3 wk)	TBD	Phase 1
2.2	**Signed capsule bundles and admission** (G1)	Canonical bundle-CID format; publisher signature record; trust-policy store (which DIDs, which roles); recursive dependency check by CID + SHA-256; install and admission receipts; publisher root key with rotation (#213)	Every launch path (local tree, CID fetch, Carrier) refuses an unsigned or tampered bundle with a clear reason. The HTTP launch route no longer records `Untrusted` for admitted capsules. Tests for each refusal.	M (3–6 wk)	TBD	Phase 1 (parallel with 2.1)
2.3	**Bus v2** (#181)	New world `elastos:bus@2`: async `providers.invoke` with `stream<u8>` bodies; an `events` stream for resident components; a render-only `ui` channel; `clock` and `random` as Bus calls (granted and audited); `lifecycle.start/stop`. Still **no WASI files or sockets.** v1 kept with a deprecation date (P10).	A resident component receives events, streams a body both ways, and draws through the UI channel. The admission check still rejects any non-Bus import.	M (3–5 wk)	TBD	2.1
2.4	**In-capsule kit**	`elastos-guest` gains a `bus2` feature: generated bindings, the `wit-bindgen` async executor (`spawn_local`), typed helpers for `elastos://` resources, a test harness, a template, and one pinned `wit-bindgen` version. Rust `std` on `wasm32-wasip2/wasip3` only later, through a Runtime-owned virtual adapter.	`elastos init` makes a Bus v2 capsule that builds, signs, installs and runs in under 10 minutes for a new developer.	S (1–2 wk)	TBD	2.3
2.5	**First app: signage or photo viewer**	A viewer that reads `elastos://object`, plays a playlist, and (with 2.10) writes play receipts. Logic in WASM, UI render-only.	Runs on Mac, Linux x86_64 and an aarch64 NAS from the **same signed bundle**. No sockets, no host paths. Opens a content capsule named by CID.	M (2–4 wk)	TBD	2.2, 2.4

#	Milestone	What it delivers	Exit criteria	Effort [Estimate]	Owner	Depends on
2.6	**Move web-frame apps one at a time**	Proposed order, simplest and most valuable first: viewers (documents, archive manager, `gba-emulator`, `elacity-player`), then chat-room logic, people, library, marketplace, assistant; Home last. Each move takes logic into WASM and leaves UI render-only. Home loses the power to mint every app's token (#173 T2/T3).	Each migrated app passes its old journeys. Its web-frame version is deleted in the same change (P10). Home holds only delegation authority.	L–XL (about 2 to 4 wk per app)	TBD	2.5
2.7	**Confine providers** (#173 T5)	Every native provider runs under its declared authority: seccomp/landlock on Linux, sandbox profiles on macOS, its own account where needed. Providers stop receiving the device key.	A provider that tries to reach outside its declared authority is blocked, and tests show it. Decrypt and custody providers are confined first.	L (1–3 mo)	TBD	Phase 1
2.8	**dDRM provider bundle moves to an Elacity package** (G7)	The open repo keeps only generic contracts: rights question, decrypt session, custody envelope interface, availability receipts and the reserved provider slots. Elacity's contract calls, royalty constants, custody implementation and coordinator policy move to an Elacity-owned repo, shipped as signed provider capsules under Elacity's licence. The coordinator in `elastos` becomes generic.	The MIT `elastos` binary builds and passes its tests with no Elacity-specific code. The Elacity bundle installs as signed providers and the mint-buy-play journey still passes. Legal sign-off on the licence (section 10).	M (3–6 wk, mostly repackaging) plus a decision	TBD (Elacity)	2.2, 2.7, Sash's decision
2.9	**Content-addressed CDN on Carrier** (G4)	Signed network delivery and the Home "Get" flow; more than one seed (or a DHT); **retrieval receipts** (who served which bytes to whom); bandwidth caps for home devices; spot-check retrievals.	A content capsule published on one Home is fetched by CID from at least two other peers, with signed availability and retrieval receipts. Losing the seed doesn't stop discovery.	L (1–3 mo)	TBD	2.2
2.10	**Metering receipts (software-signed)** (G5)	A metering provider. A receipt schema: capsule CID, content CID, device DID, time window, counter, server nonce. Batched, Merkle root anchored on Base by the Elacity bundle. Fraud controls: nonces, sampling, delayed payout.	Receipts from a test fleet roll up into a root on a Base test network that a third party can verify. Marketed as **"signed and auditable"**, not "unfakeable", until Phase 3.	L (1–3 mo)	TBD	2.5, 2.8, 2.9

#	Milestone	What it delivers	Exit criteria	Effort [Estimate]	Owner	Depends on
2.11	**`elastos-slim` profile**	A build without the gateway web UI and desktop browser stack, for appliances. A size and dependency budget in CI. Authority decisions moved from `elastos-server/src/api/` into the kernel crate (G3).	A slim aarch64 build boots the signage capsule on a NAS board. CI fails if the trusted-code line budget grows.	M (2–6 wk)	TBD	2.1

Rough total [Estimate]. 2.1 to 2.5 (first real WASM app) is about **6 to 10 weeks** of focused work after 0.8.0. The whole phase, with app migration, confinement, CDN and metering, is about **6 to 9 months**, depending on people and how many apps move.

6.3 Phase 2 exit criteria

1. At least one real product app (the signage or photo viewer) runs as a signed WASM capsule from one bundle on Mac, Linux and an aarch64 NAS.
2. Every install and launch path checks bundle CID, publisher DID and recursive dependencies.
3. Bus v2 is the documented capsule interface. Bus v1 has a removal date.
4. Home no longer holds every app's authority. Decrypt and custody providers are confined.
5. The MIT runtime builds with no Elacity-specific code. The Elacity dDRM bundle installs as signed providers.
6. Content fetched by CID from more than one peer, with receipts.
7. Software-signed play receipts anchored on a test network.

6.4 Phase 2 risks

Risk	Why it matters	Mitigation
Component-model async in Wasmtime is new and still getting security advisories (e.g. GHSA-32h6-97mm-8q3c, fixed in 48.0.4/49.0.2)	A sandbox bug is a security bug	Stay on 48 LTS patch releases. Keep fuel, memory and epoch limits.
The UI channel design is unknown	A WASM app could slide back into "just a web page"	Start with a simple render-only surface for the signage viewer. Design from that.
App migration regresses the product	Users notice breakage, not architecture	One app at a time, old journeys as the gate, the old version deleted only when the new one passes
`wit-bindgen` version skew (wit-bindgen #1305)	Two components with different executors can break	Pin one version in the kit
dDRM extraction is a legal question, not only code	Code already published under MIT stays MIT for those copies	Sash's decision plus legal advice before 2.8 (section 10)
Team bandwidth while 0.8.x and the expo compete	Phase 2 starves	Phase 2 starts only after Phase 1 exit. Expo items stay separate until Sash approves them.

7. Phase 3: trusted hardware ("baked on the chip")

7.1 What "baked on the chip" realistically means

Rong said: burn the OS into firmware for instant-on, so only capsules run [Meeting, 19 Oct §2A]. There are three ways to read that:

Reading	What it is	Verdict
A. Custom silicon	Our own chip with the Runtime in ROM or hardwired logic	**Not now.** It would cost tens of millions of dollars and years [Estimate], and a bug in ROM can't be patched. Only worth it at very large volume with a stable design. Revisit only if a chip partner offers it at their own cost and we have a frozen, audited trusted base.
B. Runtime in boot firmware (UEFI/BIOS)	Ship `elastos` inside each vendor's firmware	**Not as the first step.** Every board vendor has its own firmware. It's a per-vendor deal, and updates get harder.
C. Hardware root of trust plus a measured Runtime	The chip's fused boot key checks a signed boot chain. The Runtime's trusted base is measured at boot. The device key lives in a TEE or secure element and only signs for the measured Runtime.	**Recommended.** It gives the same guarantee users care about ("only genuine ElastOS and signed capsules run here, and the device can prove it") using hardware that partner chips already have. Steve's company says all their chips support TrustZone [Meeting, 18 Oct §1].

So **"baked on the chip" means: the chip's own root of trust vouches for our measured Runtime, and the Runtime vouches for each capsule by CID and signature.** Instant-on comes from a small, signed Linux image and capsules compiled ahead of time, not from ROM.

7.2 The Phase 3 stack [Proposal]

1. **Root of trust.** The SoC BootROM checks a signed bootloader against a key hash fused into one-time-programmable memory (RK3588 supports this; RSA, one key hash, no rotation [Web] https://pengutronix.de/en/blog/2026-06-19-rk3588-secure-boot.html).
2. **Fast boot.** U-Boot SPL in falcon mode loads a signed kernel directly. Published embedded-Linux results show login in under a second on i.MX6 [Web] https://imxdev.gitlab.io/video/tutorial/Booting-iMX6-under-one-second/. Our target is a picture on screen in about 1 to 2 s [Estimate, to be measured].
3. **Read-only system.** A dm-verity root filesystem holding `elastos-slim` and base capsules, in **A/B slots** fed by our existing signed updater (`elastos-server/src/update_controller.rs`, installed-release receipts).
4. **Measured Runtime.** Hashes of each boot stage and of the Runtime's trusted base are recorded in the TEE (OP-TEE on TrustZone; upstream supports `rockchip-rk3588` [Web] https://optee.readthedocs.io/en/stable/general/platforms.html) or, on boards without a usable TEE, the device key lives in a **secure element** such as the Microchip **ATECC608** (keys generated inside and never exported; P-256 only [Web] datasheet linked in `rtos-recommendation.md` §4.3).
5. **Identity bridge.** Our device identity is Ed25519, and the ATECC608 is P-256. At provisioning the SE's P-256 key **endorses** the Runtime's Ed25519 device key. On Base, P-256 signatures are cheap to check through the **RIP-7212 P256VERIFY precompile**, live since the Fjord upgrade (10 Jul 2024) [Web] https://docs.base.org/specifications/base-protocol/execution/precompiles.
6. **Keys and counts.** dKMS releases key shares only to that attested device key (section 9). Play receipts are signed inside the TEE or SE with a monotonic counter.

7. **Cameras via the NAS.** Cameras (64 to 128 MB of RAM) keep their own firmware and act as clients of a nearby NAS that runs the Runtime. A small companion MCU (sensor, power) talks to the SoC through a typed provider, never hosting capsules.

7.3 Milestones

#	Milestone	Exit criteria	Effort [Estimate]	Owner	Depends on
3.1	**Board choice and hardware questionnaire.** Ask partners for exact SoC, RAM, flash, TEE/OP-TEE status, secure element, fuse process, and whether the ElastOS frame can use a 64-bit chip. Pick one **RK3588-class** NAS board first, then one 64-bit frame board.	Written answers and two chosen boards with dev units in hand.	S	TBD (Sash for partner talks)	Sash's go on hardware partners
3.2	**Appliance image**: Buildroot or Yocto, falcon-mode boot, signed kernel, dm-verity root, A/B updates through our updater, `elastos-slim` as the first service, capsules AOT-compiled at install	Power-on to first picture measured and recorded on the NAS board. A bad update rolls back by itself.	L (6–10 wk)	TBD	2.11, 3.1
3.3	**Verified and measured boot** with a test key, then a production key ceremony (HSM, offline, two people)	An unsigned kernel or rootfs refuses to boot. Boot measurements are readable from the TEE or SE. Fusing is done only on test units until the ceremony is signed off.	M–L	TBD	3.2
3.4	**Device key in TEE or SE; keys sealed to the attested device**	The decrypt provider runs its key rebuild inside OP-TEE (or holds a key bound to the SE). dKMS nodes accept a release only with a valid attestation for an allowed Runtime measurement.	L	TBD (Elacity dKMS lead for the node side)	3.3, 2.8, dKMS launch gates
3.5	**Attested play counts**	Receipts carry the Runtime measurement, capsule CID and SE counter, are signed in hardware, roll up into a Merkle root, and a Base contract verifies the P-256 signature (RIP-7212). A replayed or edited receipt is rejected.	L	TBD (Elacity)	2.10, 3.4
3.6	**Partner boards and frames**	The same image and capsules run on a second vendor's board. A frame SKU ships with the slim image.	M per board	TBD	3.2–3.5
3.7	**Cameras via the NAS**	A camera feed reaches a capsule on the NAS through a typed provider, with no camera-side Runtime.	M	TBD	3.6
3.8	**Research only**: Wasmtime `no_std` + Pulley for 32-bit frames; seL4 as the long-term "small verified kernel"; custom silicon criteria	A short written go/no-go for each	S each	TBD	-

Phase 3 exit: one partner board boots our signed image in a measured way, plays a signed capsule within the target time, receives content keys only after attestation, and produces hardware-signed play receipts that a Base contract verifies.

7.4 Phase 3 risks

Risk	Mitigation
Vendor board support is often old, downstream kernels	Pick boards with upstream support (RK3588 has upstream OP-TEE). Budget for a kernel port.
One fused key with no rotation (RK3588); fusing is irreversible	HSM ceremony, two-person control, test units first, a written recovery plan
Cheap frames are 32-bit with no TEE support found (A33, RK3126)	Ask for 64-bit chips. Otherwise use an SE plus the Pulley interpreter, and promise less.
TEEs can be broken by physical attacks	The Elacity plan itself says TEEs are "defence in depth only; the threshold still rests on independent operators" [Org: `catalog/items/P-TEE.md`]. Keep the quorum.
Attestation proves which code counted, not that a human watched	Pair with sampling, audits and stake/slash. Say "attested", not "unfakeable".
Factory provisioning is a new operation	Write the provisioning runbook with the partner. Each device's endorsement record is signed and logged.

---

8. DRM packaging by content type

8.1 The general model: three separate things

Every protected item is made of three things that are named and signed separately (P13: objects, capsules and spaces stay distinct):

1. **The content capsule** (`role: content`). The encrypted assets, a manifest that lists them by hash, and the identity (hash) of its rights policy. Named by its bundle CID and signed by the creator's DID. It is inert data: it can't run.
2. **The licence and policy.** The on-chain right (an ERC-1155 Access Token from `AuthorityGateway.buyAccess`, read with `hasAccessByContentId`) plus an immutable rights policy the dKMS nodes evaluate (subscription, buy once, view or download, time windows). Today on-chain access is "one boolean per holder and KID", and signed Runtime policy owns the view/download distinction [Doc: `docs/PROTECTED_CONTENT.md` L120–122].
3. **The player** (`role: viewer`, or `app` for software). A separate, signed WASM capsule that knows how to present this kind of content. The content's manifest names it **by CID** (today it names it by name: `"viewer": "gba-emulator"` in `capsules/gba-ucity/capsule.json`).

The key itself is in none of these. It lives split across the dKMS nodes and is rebuilt only in the decrypt provider.

8.2 Recommended default: assets plus policy, player referenced by CID

Default [Proposal]: a content capsule carries **encrypted assets + manifest + policy identity**, and **references its player by CID**. It **bundles** the player only when it must work fully offline or is tied to one engine (most games).

Why:

- **One player identity (P2, decision rule 1).** A thousand films share one signed `elacity-player`. Fix a bug or a security hole once and every title benefits. If each film carried its own player, a flaw would live in a thousand copies.
- **Smaller trusted surface (decision rule 4).** The player sits next to decrypted output. Fewer, better-reviewed players mean fewer places to attack.
- **Smaller downloads and better caching.** The player is fetched once by CID and cached. Content-addressing means the same bytes are never stored twice.
- **The CID pin still gives certainty.** Because the content names its player by hash, the creator knows exactly which player will open it. There is no "latest version" surprise. A newer player needs a new content manifest or a policy that allows a set of player CIDs.
- **Bundling stays available.** When the player is bundled, it is still a separate signed capsule inside the bundle with its own CID, so admission checks it the same way.

8.3 By content type

Content	What's in the package	Player	How keys flow	Where decryption happens
Video and audio	CENC-encrypted fragmented MP4 (fMP4) segments, a media manifest, the KID (key ID) and rights-policy identity. Today `media-provider` makes the fMP4 and `protect` encrypts it and creates the 2-of-3 custody envelope [Doc: `PROTECTED_CONTENT.md` L33–37].	**Referenced by CID**: the shared `elacity-player` viewer (today a web projection; a WASM viewer in Phase 2).	Buy on Base, then a release request from the Runtime. Each of 3 custody nodes checks rights on chain; any 2 return shares encrypted to the decrypt provider's one-time key.	**Today:** the decrypt provider rebuilds the CEK in its own process and streams bounded media parts to that one viewer session [Doc: L132–151]. **Phase 3:** the rebuild runs inside the TEE. Honest caveat: decoded frames still pass through normal memory and the screen unless the SoC has a secure video path. Most cheap SoCs don't [Unverified per board].
Images (photos, art)	Encrypted image files (or tiles for very large images), a manifest, policy identity. A watermark per buyer is an option.	**Referenced by CID**: a shared image viewer.	Same as video.	**Today:** the decrypt provider. The viewer gets pixels or tiles, never the key. Elacity's `ddrm-reader` already does this on the web: it decrypts on the server and sends only a decoded "render-IR" to the browser [Org: `Elacity/ddrm-reader` README]. **Phase 3:** inside the TEE.
Documents and other non-media (PDF, ebooks, 3D, datasets)	Encrypted files, a manifest, policy identity.	**Referenced by CID**: a document or 3D viewer capsule (render-only, like `ddrm-reader`'s IR approach).	Same as video.	Decrypt provider, then a render-only viewer. A "download" right (plaintext export) is a separate, explicit policy action.

Content	What's in the package	Player	How keys flow	Where decryption happens
Games	The game's WASM code and assets, encrypted. A manifest. Policy identity. Saves are separate user objects, not part of the capsule. For emulated games (e.g. a GBA ROM) the content is the ROM and the player is the emulator.	**Usually bundled** (engine and game ship together and are tested together), **or referenced by CID** when a shared engine or emulator exists. Today's `gba-ucity` + `gba-emulator` is the reference-by-name version of this.	Licence check **at install or first launch**: the release request is bound to the game's bundle CID and the device. Extra asset packs can be released on demand.	The decrypt provider decrypts the code and hands it to Wasmtime, which compiles it once. **Honest caveat:** running code must exist as plaintext in memory; WASM can't execute encrypted code. **Phase 3:** keep the compiled cache sealed to the device key, so a copied disk won't run elsewhere. This raises the cost of copying; it doesn't make it impossible.
Software (apps)	The app capsule itself, optionally encrypted, plus a licence policy (buy once, subscription, seats).	It **is** the executable. Its own dependencies are named by CID.	Licence check at admission. For subscriptions, a time-bounded lease renewed online. Offline use is allowed until the lease ends.	Same as games: decrypt at install, compile, run in the sandbox. Phase 3 seals the compiled cache. Most software value is in updates and services, so a licence gate on updates and services often matters more than encrypting the binary.
AI agents and models	**Model weights** are a content capsule (GGUF today, with `model_content` metadata: format, licence, provenance [Code: `manifest.rs` L69–72, L136–253]). **An agent** is an app capsule (its logic) that references models and tools by CID and acts under its own `UsersAI/...` principal with the same tokens and audit as a human (P7).	The "player" is the **model provider** (native, partly confined) for weights, and the agent capsule for logic.	Weights: licence check, then release to the model provider only. Agent: normal admission. Paid inference offers are metered (2.10).	**Today:** decrypted weights must sit in the inference process's memory (and GPU). **Phase 3 and beyond:** run inference for paid models in a TEE or confidential VM. Elacity's PQ roadmap plans confidential VMs (AMD SEV-SNP, Intel TDX) for dKMS nodes [Org: `catalog/items/P-TEE.md`], which is a related but separate use.

8.4 Where decryption happens, by phase

Phase	Where the CEK exists	Who can see plaintext
Web today (Elacity website)	In the browser player or Elacity's server-side decrypt (non-media)	The user's browser; Elacity's backend for non-media
ElastOS Phase 0–2	Only inside the decrypt provider process, for one session	The viewer gets bounded plaintext output. The device owner, with effort, can read process memory.
ElastOS Phase 3	Only inside the TEE (or bound to the SE)	The TEE outputs decoded media. Keys never reach normal-world memory. Plaintext frames still reach the screen.

The video and game key flows are drawn in section 13.3.

---

9. dKMS with a quorum: what exists, and whether it stays

9.1 What I found (cited)

There are two generations of Elacity key delivery.

Generation 1: Lit Protocol (the old path, still used by the website).

- `Elacity/lit-keystore-moleculer` (private, GPL-3.0) is "a keystore service on top of Lit protocol (decentralized key management)" [Org: README]. Its design doc shows the flow: generate a random CEK, encrypt content with it, encrypt the CEK with Lit under access-control conditions, and at play time run a **Lit Action** (JavaScript stored on IPFS by CID) that calls `AuthorityGateway.hasAccessByContentId(user, contentId)`. If true, the Lit nodes return a licence holding the CEK, encrypted to the player with ECDH [Org: `docs/END-TO-END-CONTENT-PROTECTION-FLOW.md`].
- The current encoders target Lit's **v3 "Chipotle"** service (`apiUrl: https://api.chipotle.litprotocol.com`) [Org: `lib/encoders/lit-protocol/v3/ChipotleMediaEncoder.ts`]. Lit describes Chipotle as a TEE that holds root key material, permissions on Base, and Lit Actions identified by CID; generally available since 1 Apr 2026 [Web] https://spark.litprotocol.com/lit-v3-is-live-confidential-compute-and-key-management-in-one-api/. `Elacity/chipotle-functions` is a small cloud function in front of that API [Org: README].
- Elacity's public terms describe decryption as retrieving the CEK from a third-party "Secure-Decrypt Enclave" [Web] https://docs.ela.city/legal/terms-of-service.
- Elacity's PQ roadmap calls this "the old key path" and plans to **retire it**: switch off every use, revoke its credentials, and prove one route at a time that a dKMS-protected asset can never be opened through Lit (item `L14-LIT`, planned roadmap weeks 20 to 26, which works out to 8 Mar to 18 Apr 2027) [Org: `Elacity/elacity-post-quantum-roadmap`, `catalog/items/L14-LIT.md`].

Generation 2: Elacity's own dKMS quorum (in the runtime repo).

- **Shape.** AES-256 content keys, split **t-of-n** with Shamir secret sharing (the `vsss_rs` crate), each share wrapped with **hybrid X25519 + ML-KEM** (post-quantum hybrid), custody providers chosen by the Runtime, scoped decrypt sessions [Doc: `docs/GLOSSARY.md` dKMS entry; Code: `elastos/crates/elastos-protected-content-custody/src/lib.rs` L4–16, L85–86]. Suite name: `elastos-xwing-draft06-hkdf-sha256-aes256gcm/v1` [Doc: `PROTECTED_CONTENT.md` L153–154].
- **Policy.** Exactly **2-of-3 across distinct operators and failure domains**. Each node stores one share and checks the signed Runtime operation, the wallet, the KID, the full encrypted-content identity, the action, the custody epoch, the policy, **finalized rights evidence from the chain**, recipient authorization and the time window. Each node reads 2 to 5 private RPC sources and needs two agreeing finalized results [Doc: `PROTECTED_CONTENT.md` L124–138].
- **Release.** Any 2 approved nodes return **recipient-encrypted contributions**. The decrypt provider makes a one-time recipient key per operation, rebuilds the CEK in its own process, serves bounded reads to the exact viewer session, and closes everything at the end. The Runtime can carry the custody envelope but can't open the node-sealed shares [Doc: L132–151]. "Raw CEKs never pass through Runtime, Carrier, or an ordinary capsule" [Doc: `docs/CARRIER.md` dKMS over Carrier].
- **Transport.** Off-box dKMS traffic uses Carrier, while dKMS keeps its own end-to-end authorization and crypto (P15; `docs/CARRIER.md`).
- **Contracts.** Each dKMS node evaluates the immutable rights policy on its own and signs its decision before contributing [Doc: `docs/PROTECTED_CONTENT_CONTRACTS_V1.md` item 6].

- **Status.** "The canonical source path is implemented but inactive." Proven headless on an Anvil fork of Base and on a single-host, three-container simulation, not on distinct operators or real Base. External crypto review is still open [Doc: `PROTECTED_CONTENT.md` L21–24, L153–156, L277–330]. The provisional `key-provider` stays fail-closed ("fail_closed_until_dkms_backend_configured") and must never be connected to a dKMS [Code: `capsules/key-provider/src/main.rs` L130–141; Doc: `docs/KEY_PROVIDER.md`].
- **Production.** Elacity's 7 Sep 2026 security briefing says non-media decryption on base.ela.city is server-side, "backed by a 2-of-3 dKMS node quorum", and treats the quorum nodes as a separate audit [Org: `Elacity/elacity-web` `docs/SECURITY-AUDIT-BRIEFING.md`]. A June 2026 shipping report mentions aligning the Runtime to "the deployed quorum nodes" [Web] https://github.com/Elacity/pc2.net/discussions/26. **[Unverified]** whether those deployed nodes run the same Rust custody code as the repo's canonical path; the non-media reader still decrypts through a Lit Action according to its README.
- **Plan.** Elacity's PQ roadmap (297 items over 95 weeks in its source package, 304 in its tracker; week 0 = 19 Oct 2026) has a full dKMS track: release evidence from three sources, share attribution, resharing, a staging committee, separately run nodes, retiring Lit, **launch gates closed at week 38 (about 12 Jul 2027)**, then in the open-network stage: resharing **2-of-3 to 3-of-5** with no key materialised (`P-RESHARE`), **confidential-VM nodes with attestation checked at admission** (`P-TEE`, planned weeks 56 to 68), and partner operators (`P-ADMIT`). Its grade today for DKMS is F, by its own rubric [Org: `registry/milestones.yaml`, `catalog/items/P-TEE.md`, README status block].

9.2 Does dKMS stay relevant? Yes.

The quorum answers one question that no amount of sandboxing answers: **"who is allowed to get this key, and who decides?"** Without it, the key would sit with Elacity (a central server) or in the file (no protection). The new architecture changes **where the key goes** and **what evidence is needed**, not whether we need a quorum.

Phase	What dKMS does	What changes
Web today	Lit (being retired) and a 2-of-3 quorum behind server-side decrypt	Retire Lit (L14-LIT). One key path (P10).
Phase 0–1	Canonical 2-of-3 path in source; inactive in the installed product	Finish the installed proof and the atomic cutover that removes the provisional surfaces (`PROTECTED_CONTENT.md` "Remaining work").
Phase 2	Same quorum. The Runtime-side client moves into the Elacity provider bundle.	Release requests bind the **content bundle CID** and, for games and software, the **player or app bundle CID**. The recipient is the decrypt provider's one-time key, proven to belong to an **admitted** Runtime (signed release + signed capsule). Policy can name allowed player CIDs. [Proposal]
Phase 3	Same quorum, stricter evidence	Shares are released only to a recipient key that lives **inside the TEE** (or is endorsed by the SE), with an **attestation** for an allowed Runtime measurement. Threshold release stays keyed to the **on-chain licence** in the BUSL contracts. Nodes check the P-256 endorsement off chain; the chain can check it with RIP-7212 when needed. Offline devices (frames) get **time-bounded leases** sealed to the device. [Proposal]
Open network	3-of-5, partner operators, TEE-hosted nodes (Elacity PQ roadmap S5)	Device attestation (ours) and node attestation (theirs) meet: a genuine node releases only to a genuine device.

What I'm guessing, clearly marked: binding releases to bundle CIDs and allowed player CIDs, TEE-sealed recipients, and leases for offline frames are **my proposals**. They are not in the repo or the PQ roadmap I read. The PQ roadmap's `P-TEE` item is about protecting **node** shares from **node** operators, not about consumer devices.

9.3 How the pieces line up (Phase 3)

```
 creator ──encrypt──▶ content capsule (bundle CID) ──publish──▶ Carrier / IPFS (anyone can cache
it)
    │
    └──split CEK 2-of-3, shares sealed to nodes──▶ dKMS node A | node B | node C
                                                    │ each checks: licence on Base
(hasAccessByContentId),
                                                    │ rights policy, epoch, freshness, device
attestation
 buyer ──buyAccess on Base──▶ Access Token          ▼
 buyer device (measured Runtime, key in TEE/SE) ◀── any 2 shares, encrypted to the TEE recipient
key
    └── TEE rebuilds CEK ▶ decodes ▶ player capsule (by CID) shows it ▶ TEE signs play receipt
```

---

10. The licence and layer split

Intent (from Sash via Bella, 19 to 20 Oct): the ElastOS Runtime is MIT and goes to the Elastos DAO. Elacity's dDRM, meaning the provider bundle plus the smart contracts, is Elacity's, with the contracts under a time-delayed (BUSL-style) licence.

Layer	Licence	Owner	Contents
ElastOS Runtime	MIT [Code: `LICENSE`; `elastos/Cargo.toml` `license = "MIT"`]	Elastos DAO (about a $3M contract, per the conversation)	Kernel, capability tokens, Bus, Wasmtime host, Carrier, content plane, updater, generic provider slots and generic protected-content contracts (rights question, decrypt session, custody envelope interface, availability receipts)
Elacity dDRM provider bundle	Elacity's choice (time-delayed) [Proposal]	Elacity	Custody client and node, decrypt implementation, Elacity contract calls, royalty logic, settlement, metering and proof-of-play, shipped as **signed provider capsules** that any Runtime can admit by policy
Elacity protocol contracts on Base (8453)	**BUSL-1.1** [Org: `Elacity/v3-drm-protocol` `LICENSE`: "Licensor: Elacity, LLC ... Elacity Protocol Contracts identified by the SPDX-License-Identifier: BUSL-1.1 in their source files"]	Elacity	AuthorityGateway, CentralStorage, EventHub, RoyaltyTradeGateway

What the runtime repo does today [Code]. The contract source is **not** in the MIT repo; only calls, selectors and constants are. `capsules/chain-provider/src/main.rs` L40–56 holds the creator mode: the **creator's 950 of 1000 royalty-share units (95%) are sent in each mint** (`PROTECTED_CONTENT_CREATOR_ROYALTY_SHARE_UNITS = "0x3b6"`), while the **protocol owner's share is minted by the contracts from** `CentralStorage.protocolShares()`, which reads 50 (5%) on the deployed Base config, so the client mirrors the on-chain value rather than setting it. Access reads

use `AuthorityGateway.hasAccessByContentId(address,bytes16)` and `CentralStorage.ipReference(bytes16)`; `buyAccess` selectors are listed in `docs/PROTECTED_CONTENT.md` L101–118.

What has to change (milestone 2.8). Move the Elacity-specific parts (contract calls and constants, custody implementation, coordinator policy, the five protected-content crates where Elacity-specific) into an Elacity repo. Keep only generic contracts in the open Runtime. **Caution:** code already published under MIT stays MIT for the copies people already have. A time-delayed licence can apply only to new Elacity-owned code from the point it moves. This needs a lawyer's view before the move.

---

11. Risks, open decisions and metrics

11.1 Top risks across the plan

#	Risk	Effect	Mitigation
R1	Trying to do Phase 2 and the expo/partner work at the same time	Both slip	Phase 2 starts after Phase 1. Partner items need Sash's explicit go.
R2	"Unfakeable play counts" is promised before Phase 3	Credibility loss with Steve and advertisers	Say "signed and auditable" in Phase 2, "hardware-attested" only on boards with a TEE or SE (section 7.4)
R3	32-bit frames	Slow capsules, no TEE	Ask for 64-bit chips (3.1)
R4	Sandbox or engine bugs in new Wasmtime async code	Escape from a capsule	LTS patch releases, limits, provider confinement as a second wall
R5	dKMS not yet independently reviewed; nodes all run by Elacity at launch	Overclaiming decentralisation	Follow Elacity's own claims ladder ("operated by Elacity" until partners join) [Org: `registry/milestones.yaml`]
R6	Licence change on already-MIT code	Legal dispute or confusion	Legal review before 2.8; only new code is relicensed
R7	Trusted base doesn't shrink	Phase 3 measures a 200k-line binary	2.11 CI budget; move authority into the kernel crate
R8	Single seed for discovery	Outage stops content and chat	2.9 adds seeds or a DHT. Boson is one option, still awaiting Sash's go.
R9	Key ceremony mistakes (fusing, lost keys)	Bricked fleet or compromised updates	HSM, two people, test units first

11.2 Open decisions for Sash

1. **Approve Phase 2 starting after 0.8.0**, with 2.1 and 2.2 first. (No Cursor spend until the plan renews; this is a plan, not a start.)
2. **First partner board:** an RK3588-class NAS? And will partners build the ElastOS frame on a **64-bit** chip?
3. **dDRM move (2.8):** approve moving Elacity-specific code out of the MIT repo, and the licence for the provider bundle.
4. **Play-count promise:** "auditable" now, "hardware-attested" later, as written here?
5. **Player default:** agree "player referenced by CID, bundled only for offline or games"?
6. **dKMS binding:** agree that releases should bind content and player bundle CIDs (Phase 2) and device attestation (Phase 3), and pass this to the Elacity dKMS lead to fit into the PQ roadmap?

7. **On-hold items** (Boson, NAS/expo, slim frame builds, camera relay, Rong Chen's signed-dependency work with Dr. Cao): which, if any, join this plan, and when?
8. **Custom silicon:** confirm "not now" (section 7.1).

11.3 Metrics

Metric	Today	Phase 2 target	Phase 3 target
First-party apps running as signed WASM capsules	0 of 20	at least 1 (signage/photo), then most viewers	all apps except Home-specific shell parts
Install/launch paths that check bundle CID + publisher DID	0 (release-level only)	all	all
Providers confined to declared authority	1 partly (model)	decrypt, custody, then all	all, plus measured
Trusted authority code (lines)	about 9.9k kernel crate inside about 200k binary	budget set in CI and falling	measured at boot
Seeds or discovery sources	1	at least 3 or a DHT	same
Time from power-on to first picture	n/a (desktop app)	n/a	about 1–2 s target on the NAS board
Content keys released only to attested devices	n/a	n/a (admitted Runtime only)	100% on Phase 3 boards
Play receipts	none	software-signed, anchored on a test network	hardware-signed, verified on Base
Key paths in use	Lit + quorum (web); provisional (Runtime)	quorum only	quorum with attestation

12. The single roadmap table

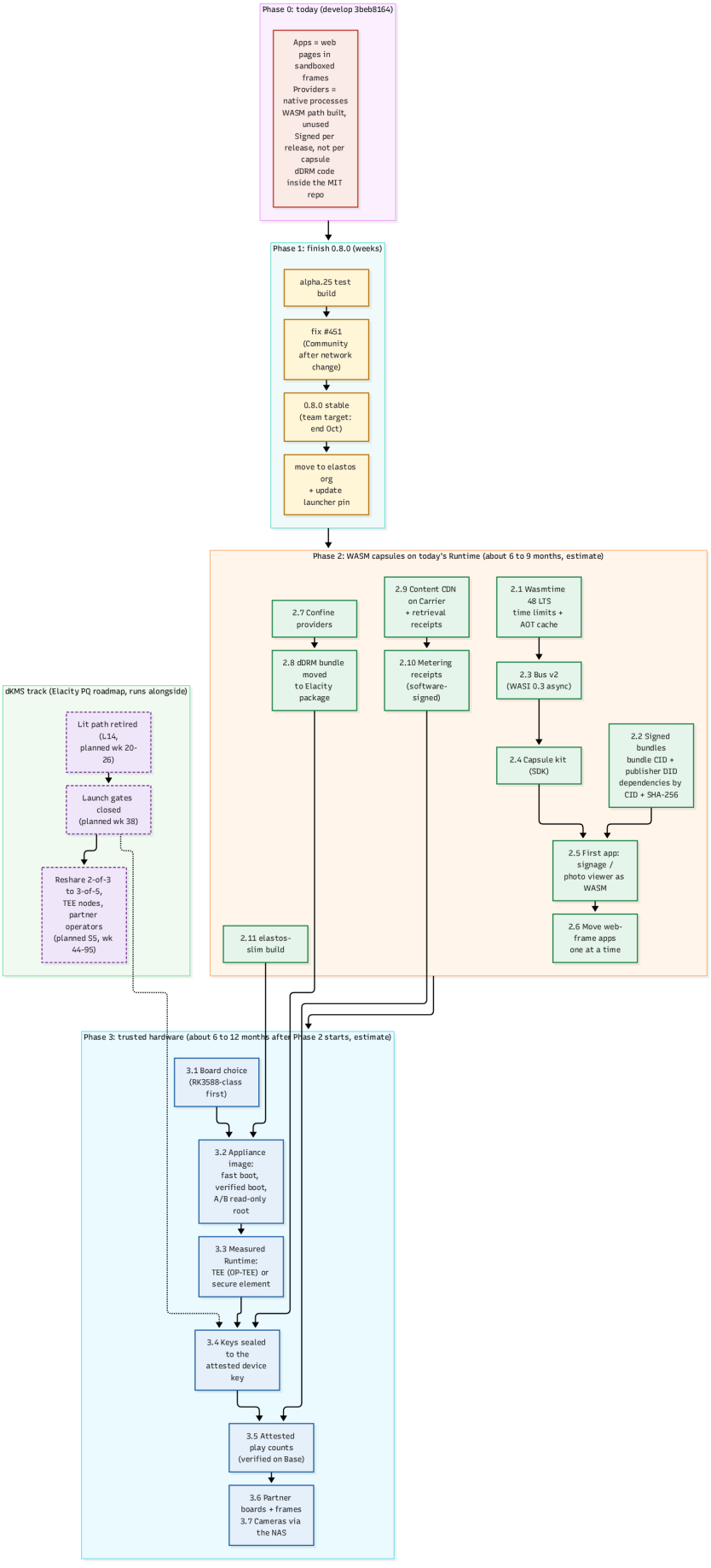
Phase	Milestone	Effort [Estimate]	Depends on	Exit in one line
1	1.1 alpha.25	S	-	published on test channel
1	1.2 fix #451	S	-	Community recovers after a network change
1	1.3 0.8.0 stable (team target end Oct)	S	1.1, 1.2	signed release out
1	1.4 move to `elastos` org	S	1.3	redirects and updater work
1	1.5 launcher pin	S	1.3, 1.4	launcher installs 0.8.0
2	2.1 Wasmtime 48 LTS + limits + AOT cache	S–M	Phase 1	conformance passes; runaway stopped
2	2.2 signed bundles + admission + dependency check	M	Phase 1	every path refuses unsigned or tampered
2	2.3 Bus v2 on WASI 0.3 async	M	2.1	resident, streaming, UI channel
2	2.4 capsule kit	S	2.3	new capsule in under 10 minutes
2	2.5 first app: signage/photo viewer	M	2.2, 2.4	one bundle on Mac, Linux, NAS

Phase	Milestone	Effort [Estimate]	Depends on	Exit in one line
2	2.6 migrate web-frame apps	L–XL	2.5	each app moved, old one deleted
2	2.7 confine providers	L	Phase 1	out-of-scope access blocked
2	2.8 dDRM bundle to Elacity package	M + decision	2.2, 2.7	MIT build has no Elacity code
2	2.9 CDN on Carrier + retrieval receipts	L	2.2	fetch by CID from 2+ peers
2	2.10 metering receipts (software)	L	2.5, 2.8, 2.9	roots verifiable on a test chain
2	2.11 elastos-slim + core budget	M	2.1	slim build on a NAS; CI budget
3	3.1 board choice	S	Sash's go	two boards in hand
3	3.2 appliance image	L	2.11, 3.1	boot time measured; A/B rollback
3	3.3 verified + measured boot	M–L	3.2	unsigned image refuses to boot
3	3.4 keys sealed to attested device	L	3.3, 2.8, dKMS gates	nodes require attestation
3	3.5 attested play counts	L	2.10, 3.4	Base contract verifies receipts
3	3.6 partner boards + frames	M each	3.2–3.5	second vendor works
3	3.7 cameras via NAS	M	3.6	camera feed to a NAS capsule
dKMS	Lit retired (Elacity `L14-LIT`)	L (their estimate: 6 engineer-weeks)	Elacity PQ roadmap	planned weeks 20–26 (8 Mar–18 Apr 2027)
dKMS	launch gates closed (Elacity `M-LAUNCH`)	-	Elacity PQ roadmap	planned week 38 (about 12 Jul 2027)
dKMS	3-of-5, TEE nodes, partners (Elacity S5)	XL	Elacity PQ roadmap	planned weeks 44–95 (Aug 2027–Aug 2028)

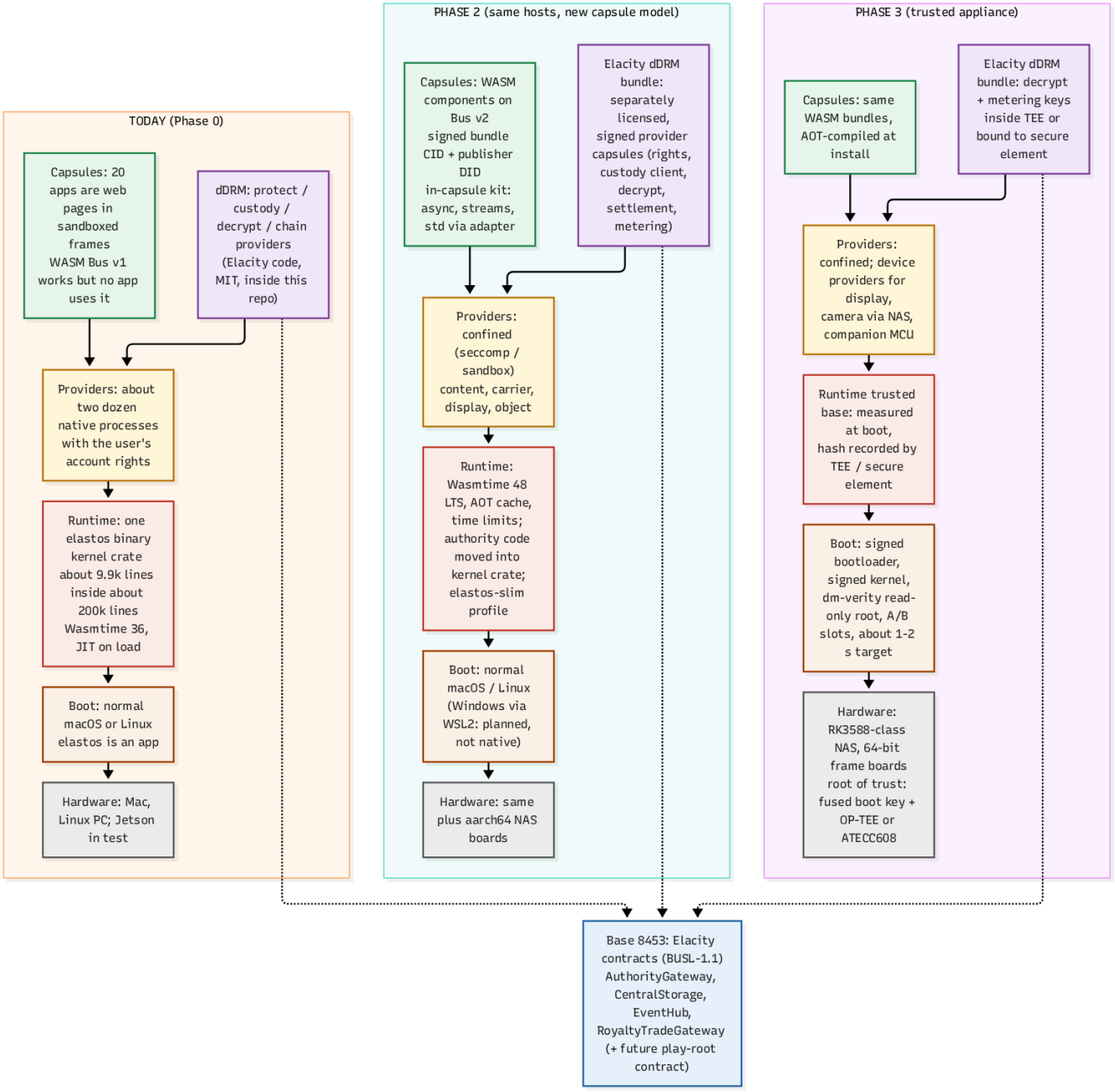
13. Diagrams

Rendered PNGs sit next to this file in `/workspace/research/prd/`. Sources are in `diagrams/*.mmd` and in the markdown version of this document.

13.1 Phase roadmap

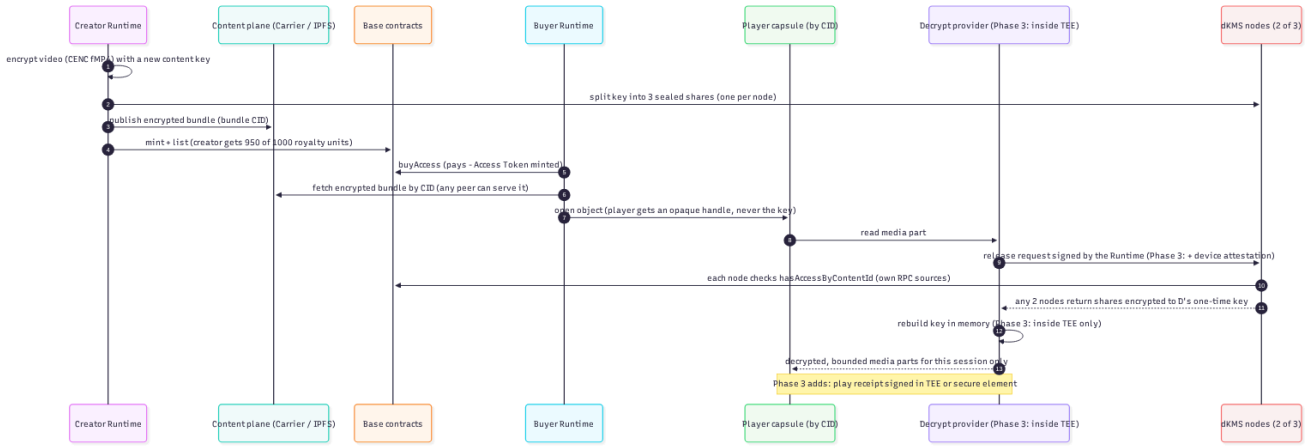


13.2 Layered architecture: today, Phase 2, Phase 3

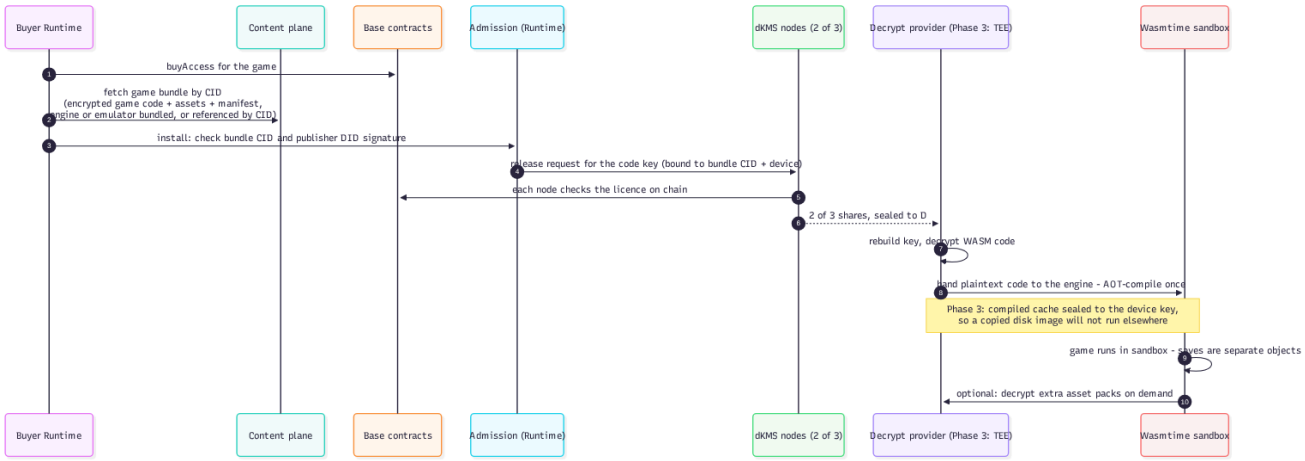


13.3 Key and DRM flow with dKMS

Video (today's canonical source path; Phase 3 additions noted):



Game (proposed; nothing here is built for games yet):



---

14. Not verified, and where my guesses are

- **0.8.0 dates and the `elastos` org move** come from the team's plan as relayed in our conversation, not from the repo. The launcher's alpha.22 pin is from the conversation; I couldn't find the pin with GitHub code search.
- **Production dKMS:** I don't know whether the "2-of-3 dKMS node quorum" behind base.ela.city's server-side decrypt runs the same Rust custody code as the runtime repo's canonical path. The private sources I read describe both a Lit path and a quorum.
- **All effort sizes and phase durations** are my estimates.
- **Phase 2 manifest v2, the signature record, release binding to bundle CIDs and player CIDs, TEE-sealed recipients and offline leases** are proposals, not existing designs.
- **Hardware:** I don't know which TEE or secure element Steve's frame and NAS boards have. Frame CPU classes (ARMv7) and camera RAM come from public teardown and vendor sources cited in `rtos-recommendation.md` §2, not from Steve's boards. Boot-time targets are estimates until measured.
- **Secure video path:** whether a given SoC keeps decoded frames out of normal-world memory is board-specific and unverified.
- **Meeting content** comes from Whisper transcripts and may contain misheard names and numbers.
- **The about-$3M DAO contract** figure is from our conversation, not a document I read.

- `docs/RUNTIME_FRAMEWORK_RESEARCH.md` (the COMO decision) is on branches `flint-0.5` and `feat/shell-ui-v1`, not on develop.
- `/workspace/runtime-principles.pdf` is an older audit prompt built on the earlier 12 principles. This document follows the current 19.

15. Sources

Runtime repo (`Elacity/elastos-runtime @ 3beb8164`): `PRINCIPLES.md`; `LICENSE`; `elastos/CHANGELOG.md`; `docs/CAPSULE_MODEL.md`, `CAPSULE_AUTHORING.md`, `CAPSULE_INTERFACE_CONTRACT.md`, `CONTENT_CAPSULE_DISTRIBUTION.md`, `PROTECTED_CONTENT.md`, `PROTECTED_CONTENT_CONTRACTS_V1.md`, `KEY_PROVIDER.md`, `CARRIER.md`, `GLOSSARY.md`, `STORAGE_AND_ACCESS.md`, `WINDOWS.md`; `elastos/wit/elastos-bus-v1.wit`; `elastos/crates/elastos-common/src/manifest.rs`; `elastos-runtime/src/capability/*`, `provider/bridge.rs`, `provider/registry.rs`, `capsule/fetched_launch.rs`; `elastos-compute/src/providers/component.rs`; `elastos-server/src/carrier.rs`, `crypto.rs`, `update_controller.rs`; `elastos-protected-content-custody/src/lib.rs`; `capsules/chain-provider/src/main.rs`, `capsules/key-provider/*`, `capsules/drm-provider/src/main.rs`, `capsules/elacity-player/capsule.json`, `capsules/gba-ucity/capsule.json`; `.github/workflows/release-package.yml`; issue #451.

Elacity org (read only): `Elacity/v3-drm-protocol` `LICENSE`; `Elacity/lit-keystore-moleculer` README, `docs/END-TO-END-CONTENT-PROTECTION-FLOW.md`, `docs/ELACITY-2010-TECHNICAL-OVERVIEW.md`, `lib/encoders/lit-protocol/v3/ChipotleMediaEncoder.ts`; `Elacity/chipotle-functions` README; `Elacity/ddrm-reader` README; `Elacity/elacity-web` `docs/SECURITY-AUDIT-BRIEFING.md`; `Elacity/elacity-post-quantum-roadmap` README, `docs/GLOSSARY.md`, `registry/repos.yaml`, `registry/milestones.yaml`, `catalog/items/DKMS-STAGE.md`, `L14-LIT.md`, `P-TEE.md`, Technical Plan Vol C outline and pages 4, 5, 10.

Web: https://docs.ela.city/exchange/drm; https://docs.ela.city/legal/terms-of-service; https://github.com/Elacity/pc2.net/discussions/26; https://spark.litprotocol.com/lit-v3-is-live-confidential-compute-and-key-management-in-one-api/; https://github.com/LIT-Protocol/chipotle; https://docs.wasmtime.dev/stability-platform-support.html; https://pengutronix.de/en/blog/2026-06-19-rk3588-secure-boot.html; https://optee.readthedocs.io/en/stable/general/platforms.html; https://imxdev.gitlab.io/video/tutorial/Booting-iMX6-under-one-second/; https://docs.base.org/specifications/base-protocol/execution/precompiles; plus the sources cited in the companion notes.

Companion notes (`/workspace/research/`): `capsule-architecture.md`, `rtos-recommendation.md` (+ `rtos-architecture.mmd/.png`), `wasm-component-model.md`, `carrier-explainer.md`, `boson-vs-carrier.md`. Meeting notes: `/workspace/meeting-2026-10-18/summary.md`, `/workspace/meeting-2026-10-19/summary.md`.